

AuraMed AI — Clinical Disease Prediction Report

Structured Multi-Dataset Disease Diagnostic Classification System

1. Executive Summary

AuraMed AI is an advanced, high-aesthetic machine learning application designed for clinical disease prediction and risk assessment. The system acts as a medical dashboard, allowing practitioners to evaluate patient clinical profiles, adjust model hyperparameters, and compare performance metrics. It provides real-time evaluations across three distinct conditions: Heart Disease, Diabetes Mellitus, and Breast Cancer.

2. Detailed Project Components

The workspace consists of the following modular files:

- **Machine Learning Pipeline (`model_pipeline.py`):** Integrates data generation, feature engineering, and model training. It generates realistic representations of the UCI Cleveland Heart Disease, Pima Indians Diabetes, and Wisconsin Breast Cancer datasets, and trains four major classification algorithms: Support Vector Machines (SVM), Logistic Regression, Random Forests, and XGBoost.
- **Flask API Backend (`app.py`):** Exposes RESTful endpoints. The server runs models on startup (port 5005), holds cached datasets in memory, and handles async requests for data preview, parameter tuning, and single-patient evaluations.
- **Glassmorphic Interface (`templates/index.html`):** A semantically structured dashboard containing sidebar parameter controls, metric cards, a Chart.js comparative canvas, cohort browser tables, and responsive patient diagnostic forms.
- **Premium Stylesheet (`static/css/style.css`):** Implements a modern dark-glassmorphic style featuring backdrop blurs, neon gradient accents, responsive grids, and micro-animations. Toggles light/dark modes seamlessly.
- **Async Controller (`static/js/main.js`):** Orchestrates frontend logic. Handles tab switching, fetches API datasets, constructs dynamic clinical inputs based on the selected disease type, updates Chart.js figures, and renders risk percentages.
- **Automated Verification Suites (`test_pipeline.py` & `verify_endpoints.py`):** Programmatic checks asserting data shapes, model serialization, API status, and low-risk vs high-risk patient prediction endpoints.

3. Cohorts & Engineered Clinical Ratios

To increase classifier decision boundary separation, the system automatically engineers custom clinical ratios on raw variables:

- **Heart Disease Cohort (Cleveland):** Analyzes 13 symptoms. Features a custom *BP-to-Cholesterol Ratio* (systolic strain relative to arterial clogging) and a *Max Heart Rate-to-Age Ratio* (evaluating cardiac performance relative to biological age).
- **Diabetes Cohort (Pima):** Evaluates 8 variables. Engineers an *Insulin-to-Glucose Ratio* (surrogate for pancreatic beta-cell sensitivity) and a *BMI-to-Age Ratio* (assessing metabolic loading over lifespan).
- **Breast Cancer Cohort (Wisconsin):** Measures 10 nuclear features. Engineers an *Area-to-Perimeter Ratio* (quantifying cell shape irregularity) and a *Compactness-Concavity Product* (measuring nuclear indentation severity).

4. Models & Algorithms Implemented

Four distinct machine learning algorithms are trained and compared in parallel for each disease condition:

- 1. Support Vector Machine (SVM):** Fits optimal decision hyperplanes. Supports RBF or linear kernels to handle non-linear separation boundaries.
- 2. Logistic Regression:** A baseline linear classifier mapping coefficients to sigmoid probabilities. Offers maximum clinical interpretability.
- 3. Random Forest:** An ensemble of decision trees. Reduces variance and overfitting, yielding robust predictions on multi-dimensional clinical data.
- 4. XGBoost / Gradient Boosting:** Sequentially optimizes weak learners using gradient descent. Achieves state-of-the-art predictive performance.

5. Technical Stack & Dependencies

Technology	Role / Usage	Key Packages
Python 3.14	Backend Computing & Modeling	Scikit-Learn, XGBoost, Pandas, Joblib
Flask	Web Framework & API Server	Werkzeug, Jinja2, HTTP Server
HTML5 / JavaScript	Frontend Interface Logic	Chart.js (v4), FontAwesome (v6)
Vanilla CSS3	Visual Presentation	Glassmorphism variables, dark/light themes

6. Programmatic Verification Results

A two-layer test suite has been successfully executed to confirm project stability:

Layer A: Pipeline Unit Tests (test_pipeline.py): Asserts that all datasets generate with correct dimensions, preprocessors/scalers fit and transform correctly, multi-model evaluation maps are filled, model files serialize successfully, and predictions evaluate correctly. **Result: 5/5 Tests Passed.**

Layer B: API Integration Tests (verify_endpoints.py): Validates that the active server (port 5005) returns index HTML, data endpoints fetch preview records and statistics for heart, diabetes, and breast cancer datasets, models retrain on custom request parameters, and single predictor inputs score correctly for high/low risk. **Result: All API routes verified successfully.**

7. Local Running Instructions

The Flask application is currently running as a background task. Follow these instructions to view the live dashboard:

1. Open your browser.
2. Navigate to: **http://127.0.0.1:5005/**
3. Use the dataset dropdown to toggle between Heart Disease, Diabetes, and Breast Cancer datasets.
4. Adjust sidebar parameters to run model retraining or submit mock patient parameters in the predictor tab to view risk explanations.

8. Project Conclusion

AuraMed AI achieves an integrated diagnostic utility. By wrapping raw clinical indicators in mathematical ratios and training advanced algorithms (SVM, Logistic Regression, Random Forest, and XGBoost), the application yields high accuracy and actionable predictive metrics. The custom medical explanation text bridges the gap between raw probability numbers and medical reasoning, supporting clinicians in risk assessment.